

BÀI TẬP

Conditional and Loop

Câu lệnh điều kiện và Vòng lặp

Quanskill
quanskill.com

Thông tin

Câu lệnh điều kiện (conditionals) và vòng lặp (loops) là hai khái niệm cơ bản nhất trong lập trình, giúp máy tính đưa ra quyết định và lặp lại các tác vụ một cách thông minh.

1. Bài tập Trắc nghiệm (Multiple-Choice Questions - MCQ)

Dựa trên các khái niệm về if/else, for/while, toán tử logic, và break/continue.

1.1. Câu hỏi

Bài tập

Câu 1: Cấu trúc điều kiện nào được sử dụng khi bạn cần kiểm tra nhiều hơn hai tình huống hoặc điều kiện khác nhau?

- A. if-else
- B. if-elif-else
- C. while
- D. for

Bài tập

Câu 2: Toán tử logic nào trong Python trả về giá trị True nếu và chỉ nếu cả hai điều kiện đều đúng?

- A. or
- B. not
- C. and
- D. elif

Bài tập

Câu 3: Mục đích chính của vòng lặp for là gì?

- A. Lặp lại một khối mã miễn là điều kiện vẫn đúng.
- B. Thực hiện các hành động khác nhau dựa trên điều kiện cụ thể.
- C. Lặp qua các phần tử trong một danh sách, chuỗi, hoặc một dãy số.
- D. Định nghĩa một hàm.

Bài tập

Câu 4: Trong một vòng lặp, câu lệnh break có chức năng gì?

- A. Bỏ qua lần lặp hiện tại và tiếp tục với lần lặp tiếp theo.
- B. Thoát khỏi vòng lặp ngay lập tức.
- C. Đảo ngược giá trị của điều kiện.
- D. Tạo ra một vòng lặp lồng nhau.

Bài tập

Câu 5: Khối mã sau sẽ in ra những giá trị nào?

```
numbers = [1, 2, 3, 4, 5]
for num in numbers:
    if num == 3:
        continue
    if num == 5:
        break
    print(num)
```

- A. 1, 2, 3, 4
- B. 1, 2, 4, 5
- C. 1, 2, 4
- D. 1, 2, 3, 4, 5

2. Bài tập Thực hành

Tài liệu cung cấp các bài tập thực hành được chia thành hai cấp độ: Cơ bản và Nâng cao.

2.1. Bài tập Cơ bản (5.1)

Bài tập

Bài 1: Kiểm tra số

Viết chương trình kiểm tra một số có phải là số dương, âm, hay bằng 0.

Yêu cầu:

- Nhập vào một số từ người dùng
- In ra thông báo tương ứng với loại số đó

Gợi ý cấu trúc:

```
# Get input from user
number = int(input("Enter a number: "))

# Check and print result
if condition1:
    print("The number is positive")
elif condition2:
    print("The number is negative")
else:
    print("The number is zero")
```

Bài tập

Bài 2: In số chẵn

Viết chương trình in ra các số chẵn từ 1 đến 10.

Yêu cầu:

- Sử dụng vòng lặp for
- Kiểm tra tính chẵn lẻ bằng toán tử modulo (%)
- In ra từng số chẵn trên một dòng

Gợi ý cấu trúc:

```
# Loop through numbers from 1 to 10
for i in range(1, 11):
    # Check if number is even
    if i % 2 == 0:
        print(i)
```

Bài tập

Bài 3: Tính tổng

Viết chương trình tính tổng của các số từ 1 đến 100.

Yêu cầu:

- Sử dụng vòng lặp for hoặc while
- Khởi tạo biến tổng ban đầu
- In ra kết quả cuối cùng

Gợi ý cấu trúc:

```
# Initialize sum variable
total = 0

# Loop and calculate sum
for i in range(1, 101):
    total += i

# Print result
print(f"Sum from 1 to 100 is: {total}")
```

2.2. Bài tập Nâng cao (5.2)

Bài tập

Bài 1: Chuỗi đối xứng (Palindrome)

Viết chương trình kiểm tra một chuỗi có phải là chuỗi đối xứng (palindrome) hay không.
Yêu cầu:

- Nhập chuỗi từ người dùng
- Không phân biệt chữ hoa chữ thường
- Bỏ qua khoảng trắng và ký tự đặc biệt
- In ra kết quả kiểm tra

Gợi ý cấu trúc:

```
# Get input and process
text = input("Enter a string: ").lower()
clean_text = ""

# Remove spaces and special characters
for char in text:
    if char.isalnum():
        clean_text += char

# Check palindrome
is_palindrome = clean_text == clean_text[::-1]

# Print result
if is_palindrome:
    print("The string is a palindrome")
else:
    print("The string is not a palindrome")
```

Bài tập

Bài 2: Bảng cửu chương

Viết chương trình in ra bảng cửu chương từ 1 đến 10.

Lưu ý: Bạn có thể cần sử dụng vòng lặp lồng nhau để giải quyết bài toán này.

Yêu cầu:

- Sử dụng vòng lặp lồng nhau
- Format đẹp cho output
- In từng bảng cửu chương một cách rõ ràng

Gợi ý cấu trúc:

```
# Outer loop for multiplication tables
for i in range(1, 11):
    print(f"Multiplication table for {i}:")

    # Inner loop for each multiplication
    for j in range(1, 11):
        result = i * j
        print(f"{i} x {j} = {result}")

    print("-" * 20) # Separator line
```

Bài tập

Bài 3: Tìm số lớn nhất

Viết chương trình tìm số lớn nhất trong một danh sách.

Yêu cầu:

- Tạo danh sách số hoặc nhập từ người dùng
- Sử dụng vòng lặp để duyệt qua danh sách
- Không sử dụng hàm max() có sẵn
- In ra số lớn nhất tìm được

Gợi ý cấu trúc:

```
# Define or input list of numbers
numbers = [3, 7, 2, 9, 1, 8, 5]

# Initialize max with first element
max_number = numbers[0]

# Loop through the list
for num in numbers:
    if num > max_number:
        max_number = num

# Print result
print(f"The largest number is: {max_number}")
```

3. Đáp án Trắc nghiệm

Đáp án

Đáp án các câu hỏi trắc nghiệm:**Câu 1: B - if-elif-else**

Giải thích: Cấu trúc if-elif-else cho phép kiểm tra nhiều điều kiện khác nhau một cách tuần tự. Khi có nhiều hơn hai tình huống cần xử lý, elif (else if) là lựa chọn phù hợp.

Câu 2: C - and

Giải thích: Toán tử logic "and" chỉ trả về True khi cả hai điều kiện đều đúng. Nếu một trong hai điều kiện là False, kết quả sẽ là False.

Câu 3: C - Lặp qua các phần tử trong một danh sách, chuỗi, hoặc một dãy số

Giải thích: Vòng lặp for được thiết kế đặc biệt để lặp qua các phần tử trong một iterable (có thể lặp được) như list, string, range, v.v.

Câu 4: B - Thoát khỏi vòng lặp ngay lập tức

Giải thích: Câu lệnh break được sử dụng để thoát hoàn toàn khỏi vòng lặp hiện tại, không thực hiện các lần lặp còn lại.

Câu 5: C - 1, 2, 4

Giải thích:

- Số 1: In ra vì không thỏa mãn điều kiện nào
- Số 2: In ra vì không thỏa mãn điều kiện nào
- Số 3: Bỏ qua vì continue được thực hiện
- Số 4: In ra vì không thỏa mãn điều kiện nào
- Số 5: break được thực hiện, thoát khỏi vòng lặp